

Method Selection and Planning

Group Number: Cohort 1, Group 11

Group Name: Y111 Studios

Group Members:

1. James Hutchinson
2. Somto Igweonu
3. Robert Kisloski
4. Sam Knight
5. Kenneth Kok
6. Ashish Kumar

This document outlines the planning phase of the development process of our game, highlighting the engineering methods and the development & collaboration tools that we have utilised to help us work effectively as a team to achieve our goals. This also includes our project plan and how this plan changed over time as we made progress.

Software Engineering Methods and Development/ Collaboration Tools

The main software engineering method is feature-driven development, where the goal is to follow the requirements and split the implementation into separate features that can be developed simultaneously. This style of development promotes quick results and emphasises good object-oriented design by aiming for each feature to contain objects designed to be flexible and applicable over a wide range of use cases. This integrates well with the Java programming language, which is a strongly object-oriented language.

The main development tool used was the Gradle project management system. It was the system used by the previous team and provided a quick and easy way to build a libGDX project as well as integrating well with GitHub Actions and not requiring much specialist programming knowledge meaning any changes to the build were easy to do.

For a version control system, Git was used. This was selected as it is a very common version control tool for software projects, is very well documented, and has many tutorials available for some of the more complex features. One of the other main features is branches, where a separate branch can be split off from a specific point and merged back in later to the main branch, which fits well with the feature-driven development process we selected. Due to this, merge conflicts should not occur or be trivial to resolve.

Expanding upon Git, we used GitHub as a way to have a remote repository, which allows for collaboration as well as maintaining a consistent state for the project among the team. GitHub also has additional tools that are built upon Git, such as pull requests and the code review system. Issues are also a feature implemented by Github, which allows each feature to be given an owner and allows them to be linked to a pull request that can be closed when the pull request has been merged. This allows for easy collaboration on the project as each team member can clone the remote repository, make local changes, and push them up onto a branch, which can be merged later and reviewed by others. This review process is important as it allows another to request changes to the code, which helps in enforcing a consistent code style across the code base.

These tools are a good fit for this project as they automate repetitive processes, which reduces the risk of errors occurring due to incorrectly applying the build process, or by pushing breaking code to the repository.

Some alternatives that were considered were GitLab in place of GitHub. This was not selected as GitHub has a simplified UI and a larger community making it easier to adopt than GitLab. The ease of use and familiarity that comes with using GitHub makes it a more suitable platform to use than GitLab.

Another consideration was whether to swap from a Gradle build to a Maven one as it has a simpler way to configure build processes by using a predefined lifecycle and uses its large

number of stable plugins to simplify build tasks, such as shading. The configuration of the project can also be done in an XML file, which provides a familiar and easy-to-read configuration as it is more declarative than Gradle. This was deemed an unnecessary change that would add significant time to the project. Maven can be slower than Gradle as well as it lacks in-built caching and parallelism and it doesn't integrate with libGDX as smoothly as Gradle.

The other software engineering method that was considered was using the agile methodology. This is an iterative approach and is similar to feature-driven development in many ways, but due to the short time frame of the project, the idea of using sprints did not seem to fit well with the time frame of the project. This entire section of development could be considered one sprint as feature-driven development is effectively the internal methodology of one sprint applied over the entire time frame of the project. Since the list of requirements was defined early on and the architecture was understood, using feature-driven development seemed like a better fit as it focused on each component of the final system, resulting in more flexible code being made.

Team Organisation

Our team was roughly split into two parts, with one working on the implementation and the other working on all the other deliverables. Initially, it was decided that two people would work full-time on implementation in order to reduce conflicts and increase code cohesion; however, this had to be adapted to allow for the other group members to contribute in order to get the implementation finished and evaluated on time. This meant relying on Github issues, branches, and pull requests in order to ensure the previously stated goals of few conflicts and high code cohesion were achieved.

Robert and Sam initially offered, and were chosen, to work on the implementation as they were the more skilled programmers and had produced many of the ideas for how to build on the previous team's project during the initial planning phase. Robert planned to focus more on the UI whereas Sam planned to work more on the data representation and logic side of the project as this was their respective areas of expertise in assessment 1 and also helped to clarify what each person was responsible for meaning it was clearer if either weren't meeting their targets. This was further clarified by making sure that any proposed additions were declared using GitHub issues so that it was easier to follow any changes.

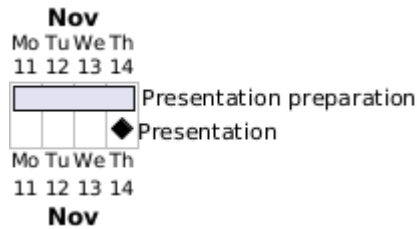
For the remaining members, the rest of the deliverables were split as evenly as possible between them, with the responsibility for the changes to the assessment 1 deliverables typically going to whomever worked on that deliverable more during the first assessment. They would also help to review any pull requests on GitHub as this served to not only provide a fresh perspective on the changes, reducing the likelihood of bugs in the code, but also to ensure that they remained knowledgeable about the implementation which helped to ensure the accuracy of the deliverables.

In general, there was no single leader of the team as that allowed for greater creativity and allowed for multiple parts of the project to progress rapidly without being slowed down by constantly needing approval. This relied on each group member checking the others' progress during team meetings to ensure that they were meeting their targets and it also meant that decisions were made by group consensus. Naturally, there were moments where more emphasis was placed on a particular group member's part of the project in which case they could be considered the 'leader' as the one who is most knowledgeable about the area; however, everyone still had their say in the final decision.

Project Plan

Week 1

This week is dedicated to preparing for and doing the presentation of our project to other teams.



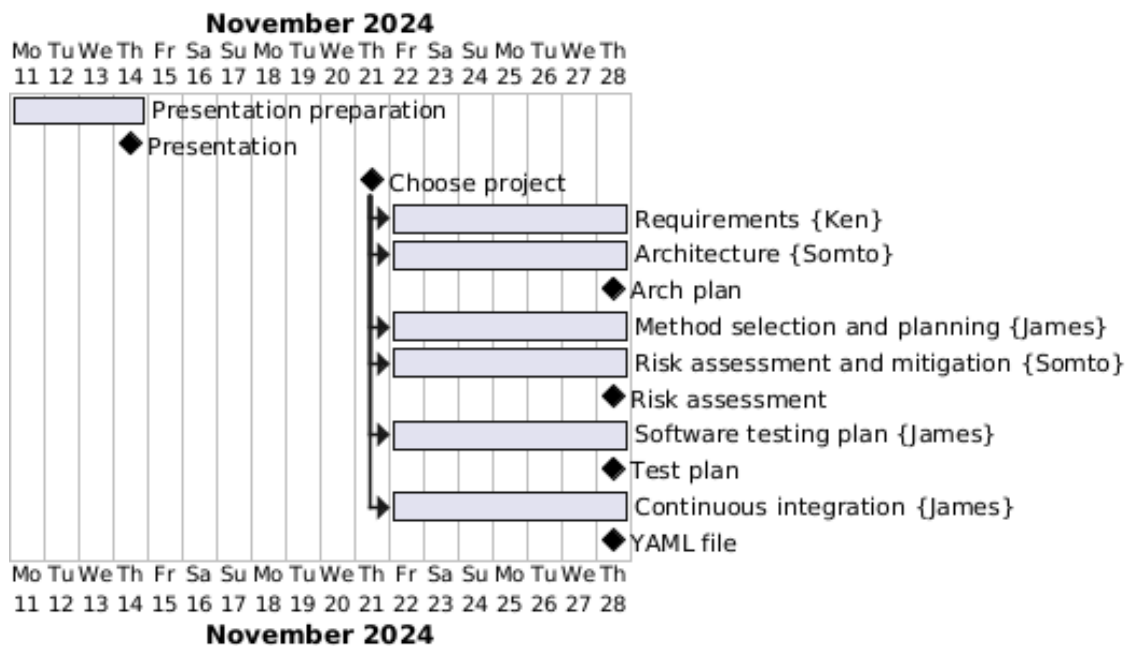
Week 2

This week is dedicated to deciding whose project we will choose for assessment 2.



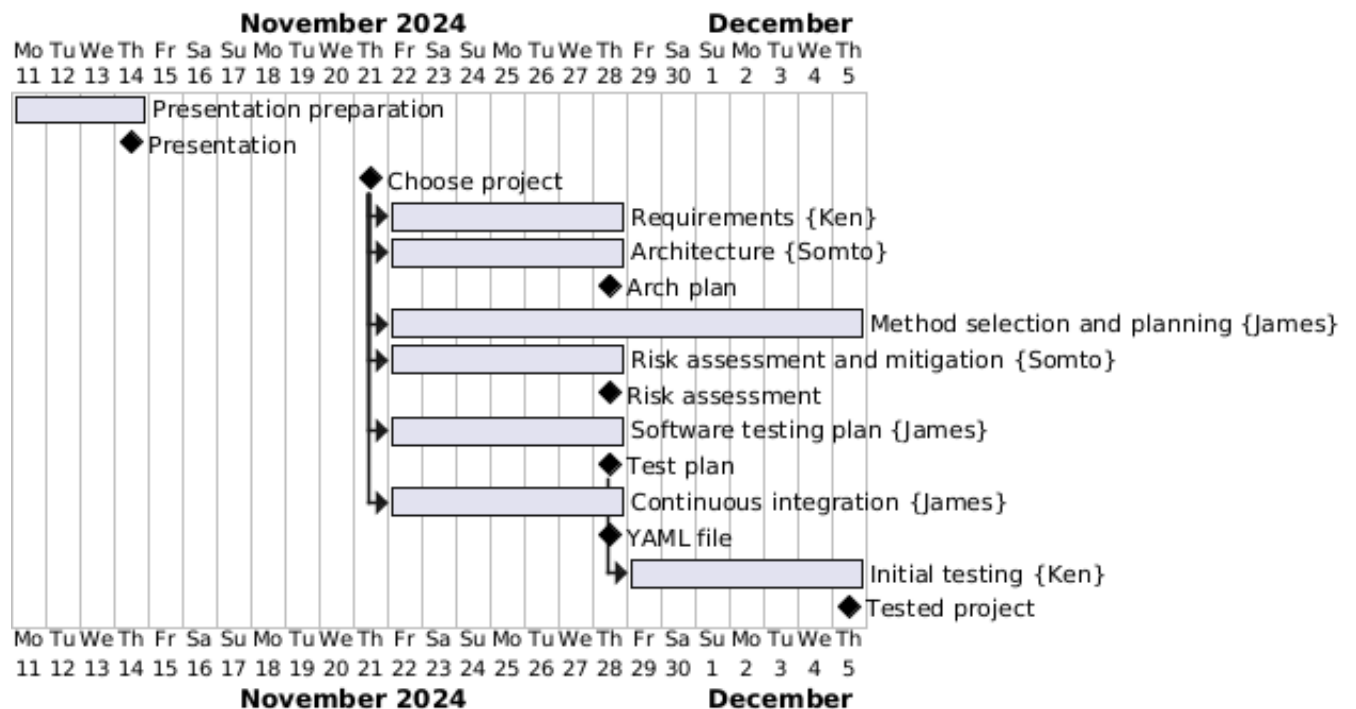
Week 3

We now have a plan as to who will be doing what section of the project and can separate the tasks for next week amongst ourselves. There are a number of tasks that have no impact on the progress of other sections so can be done on an ongoing basis whereas there are others that have specific milestones to reach. The software testing plan must be done before testing can begin and the basic testing of the previous group's code must be completed before any additional features can be added as well as a rough plan of the architecture. We also decided that we wanted a basic CI workflow that would compile the code and produce a JAR file before week 4.



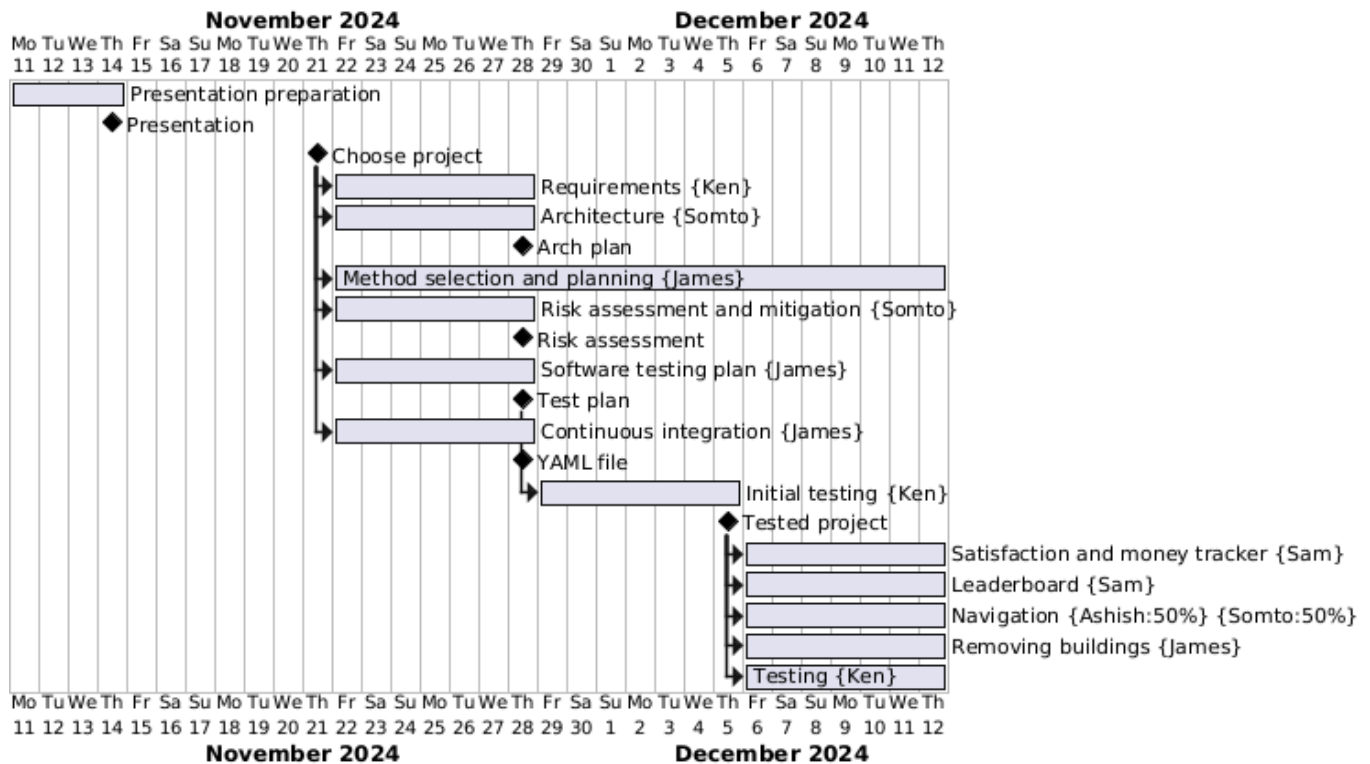
Week 4

For this week, the main focus was getting the previous team's project to an acceptable level of test coverage before we advanced. There were also many non-specific tasks not shown in the gantt chart such as adding finishing touches to some of the plans and getting familiar with the previous team's code.



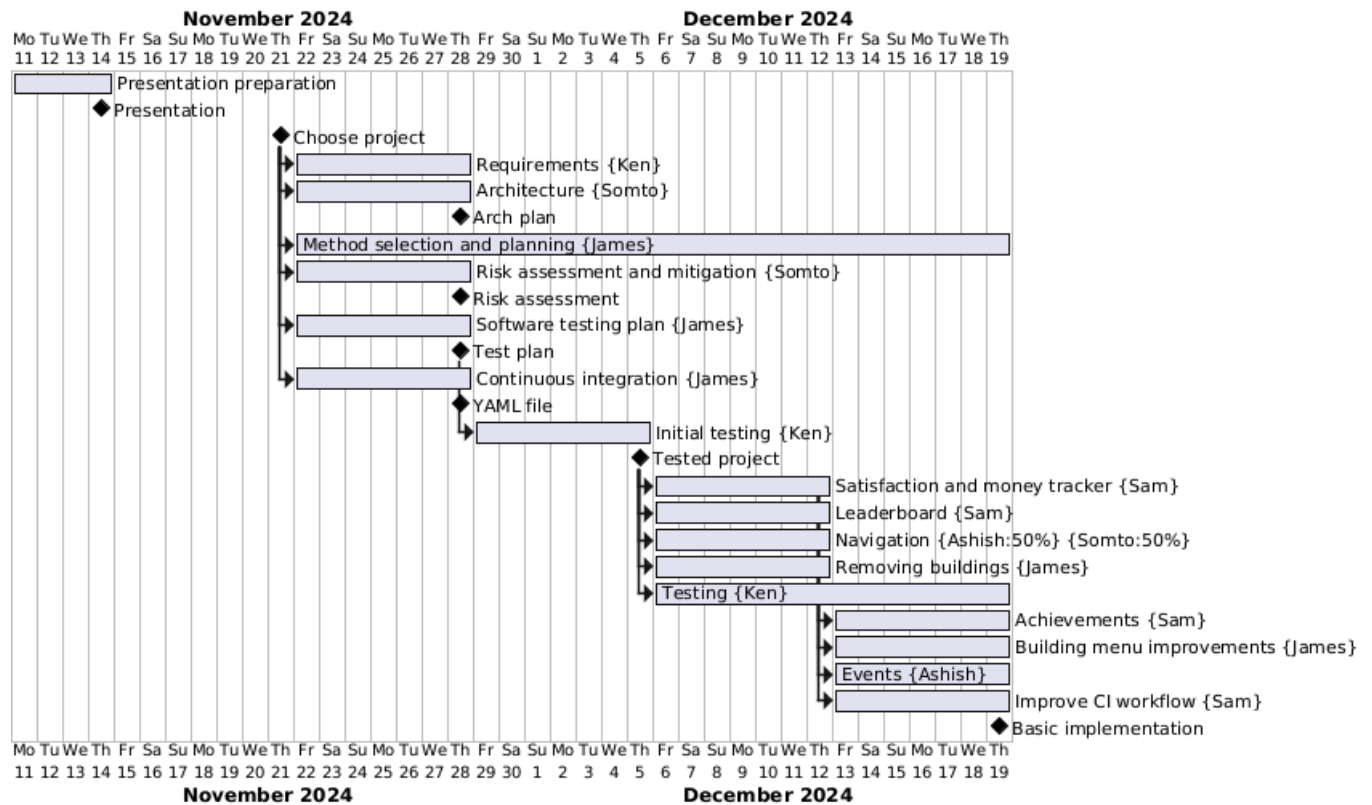
Week 5

We had originally planned to have Sam and Robert working on implementation; however, this had to be changed leading to a much more fractured plan where small sections of implementation were split between the rest of the group. It also became more important for our plans to be specific and for people to follow the plan.



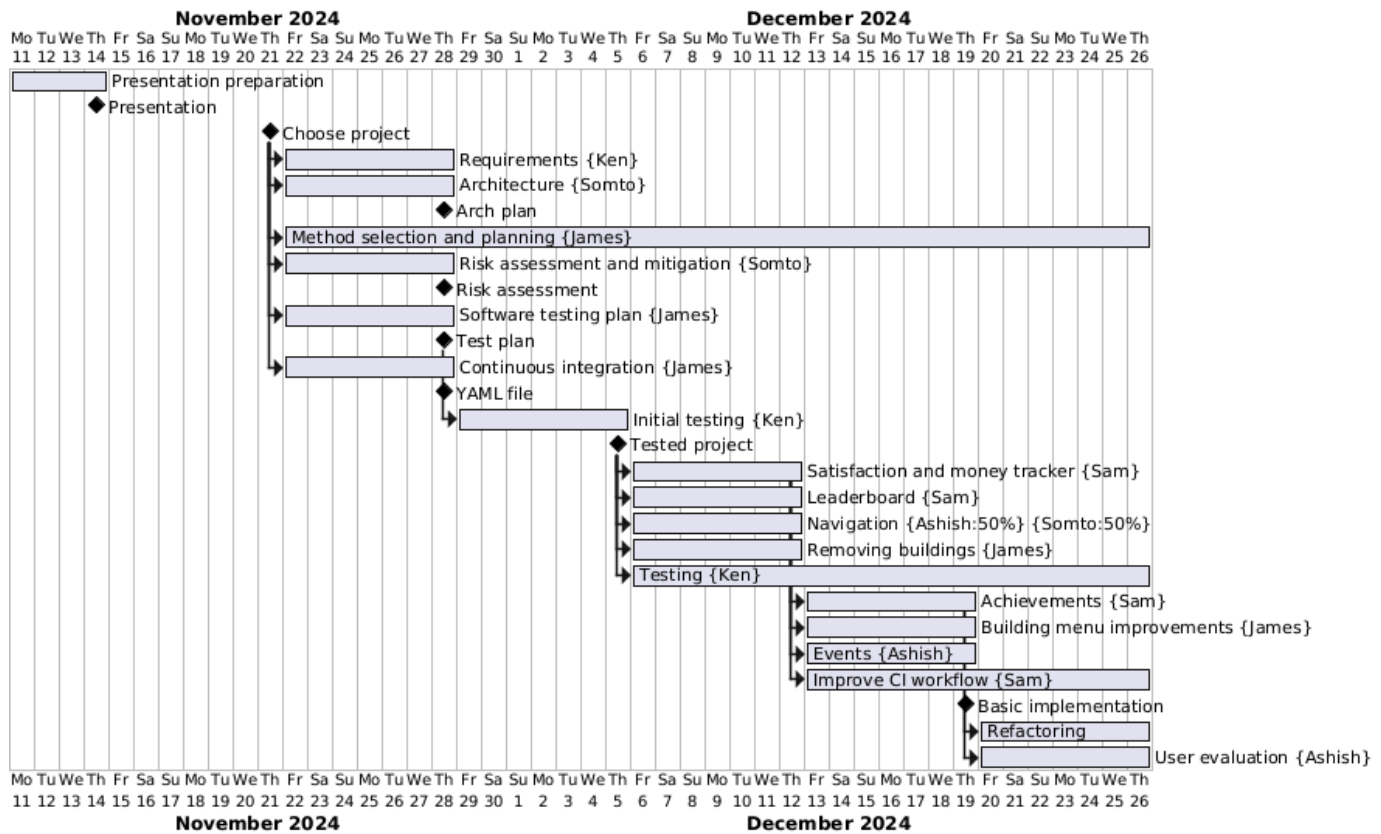
Week 6

This week started a period of increased remote working where the times when people were available to do work were less regular. It also followed a very productive week where much of the groundwork for assessment 2 was laid meaning there was a lot of refactoring and minor adjustments, mostly done by Sam and Ken, that isn't shown in the gantt chart. There was also an increase in the use of pull requests due to the remote working with many of these pull requests being reviewed by Ashish.



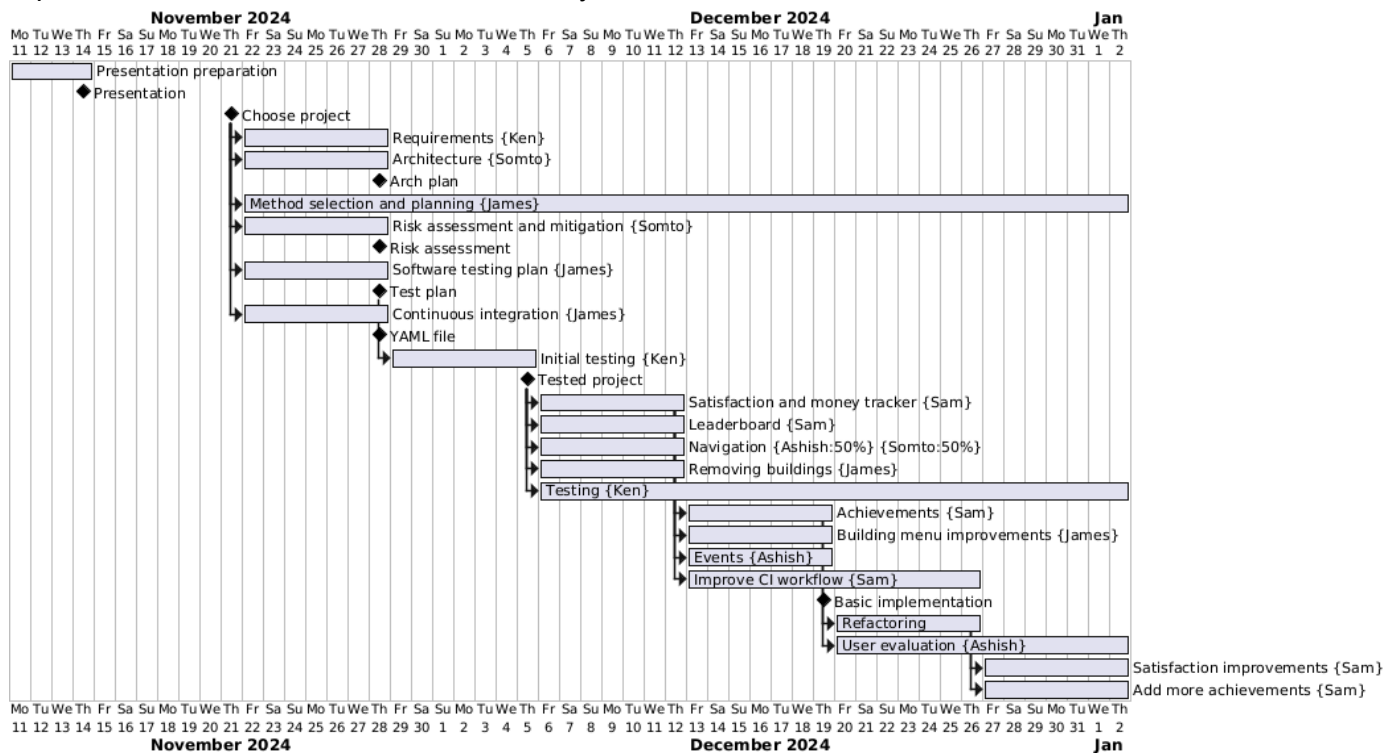
Week 7

This week marked a lower level of large changes due to the holidays; however, there was a greater focus on refactoring and generally improving the quality of the existing code. There was also much discussion about what to do next as most of the requirements had been met so the focus shifted to improving the quality and cohesion of the system.



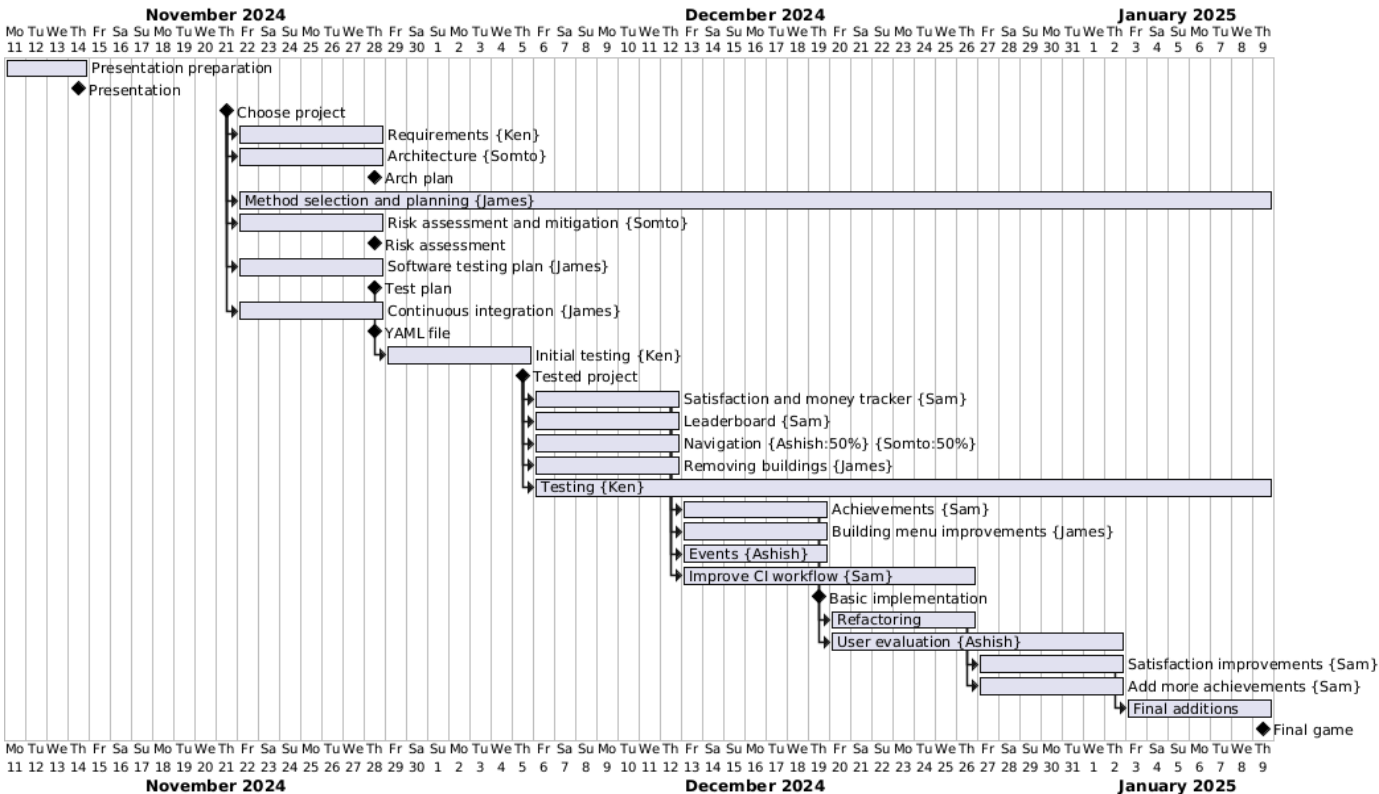
Week 8

This week saw general improvements and refactoring like last week; however, the improvements this week were more driven by the results of the user evaluation.



Week 9

This week saw the last additions to the game being made



Week 10

This week saw a review of the whole project before the final hand-in date. We also began to plan for the assessed presentation after the hand-in date.

